

Corpus Pedagógico Python v2 — SOFIA.py

Universidad Pedagógica y Tecnológica de Colombia · Ingeniería de Sistemas

Este corpus amplía el material de referencia del sistema SOFIA con guías de laboratorio, errores frecuentes documentados, preguntas frecuentes de estudiantes y rúbricas de evaluación de los cursos de Programación I y Estructuras de Datos de la UPTC.

Capítulo 1: Variables y Tipos de Datos

Guía de laboratorio — Programación I UPTC

Una variable es un nombre que referencia un valor en memoria. En Python, el tipo se infiere automáticamente según el valor asignado. Los tipos fundamentales son: int, float, str, bool y NoneType.

```
x = 10 # int precio = 3.14 # float nombre = 'Ana' # str activo = True # bool vacio = None # NoneType print(type(x)) #
```

Errores frecuentes documentados en clase:

TypeError: no se puede sumar str e int sin convertir. Ejemplo: `print('Edad: ' + 25)` genera error. Correcto: `print('Edad: ' + str(25))`

NameError: usar una variable antes de declararla. Python distingue mayúsculas: `nombre` y `Nombre` son variables diferentes.

Preguntas frecuentes de estudiantes:

¿Por qué Python no necesita declarar el tipo? Python infiere el tipo en tiempo de ejecución. Esto se llama **tipado dinámico** — la variable puede cambiar de tipo, aunque no es recomendable hacerlo.

¿Cuándo usar `int()` y cuándo `float()`? Usa `int()` cuando necesites un número entero sin decimales. Usa `float()` cuando necesites precisión decimal. Ojo: `int("3.14")` genera **ValueError** — debes hacer `int(float("3.14"))`.

Rúbrica de evaluación — Ejercicios de tipos:

Excelente (5.0): Declara variables con nombres descriptivos en `snake_case`, usa el tipo correcto, convierte entre tipos correctamente. **Bueno (4.0):** Logra el resultado pero usa nombres poco descriptivos. **Aceptable (3.0):** Funciona pero mezcla tipos innecesariamente.

Capítulo 2: Condicionales

Las estructuras condicionales ejecutan código solo si se cumple una condición. Python usa if, elif y else con indentación obligatoria de 4 espacios.

```
nota = 75 if nota >= 90: print('Excelente') elif nota >= 60: print('Aprobado') else:  
print('Reprobado')
```

```
Operador ternario: resultado = 'par' if x % 2 == 0 else 'impar'
```

Error más común: usar = (asignación) en lugar de == (comparación) dentro del if. `SyntaxError: if x = 10`. Correcto: `if x == 10`.

¿Puedo anidar if dentro de otro if? Sí, pero más de 3 niveles de anidamiento indica que el código necesita refactorización. Considera usar funciones auxiliares.

Capítulo 3: Ciclos e Iteración

Los ciclos permiten repetir un bloque de código. Python tiene for (iterar sobre secuencia) y while (mientras condición sea verdadera).

```
# Ciclo for con range – imprime 1 al 5 for i in range(1, 6): print(i) # range(inicio, fin,
paso) – fin no se incluye for i in range(0, 10, 2): print(i) # 0 2 4 6 8 # while con
condición de parada contador = 0 while contador < 5: print(contador) contador += 1 #
SIEMPRE incrementar
```

Ciclo infinito — error crítico: while x > 0: print(x) — si x nunca cambia, el programa nunca termina. Siempre verificar que la variable de control cambie en cada iteración.

break termina el ciclo anticipadamente. continue salta a la siguiente iteración. Usar con moderación — los ciclos claros no los necesitan.

¿Cuándo usar for y cuándo while? Usa for cuando sabes cuántas iteraciones necesitas (recorrer una lista, repetir N veces). Usa while cuando la condición de parada depende de algo que puede cambiar de forma impredecible.

¿Por qué range(1, 6) genera 1,2,3,4,5 y no incluye el 6? Por convención matemática de intervalos semi-abiertos [inicio, fin). Esto permite que range(0, len(lista)) recorra exactamente todos los índices válidos.

Capítulo 4: Funciones

Una función es un bloque de código reutilizable. Se define con `def`, puede recibir parámetros y retornar valores con `return`.

```
def saludar(nombre): """Retorna un saludo personalizado.""" return 'Hola, ' + nombre #  
Llamar la función resultado = saludar('Luis') print(resultado) # Hola, Luis # Parámetro con  
valor por defecto def potencia(base, exponente=2): return base ** exponente  
print(potencia(3)) # 9 print(potencia(3, 3)) # 27
```

Diferencia crítica entre `print` y `return`: `print()` solo muestra en pantalla y retorna `None`. `return` devuelve un valor que puede usarse en el programa. Si haces `resultado = funcion_con_print()`, `resultado` será `None`.

```
def doble_print(x): print(x * 2) # muestra pero no retorna def doble_return(x): return x *  
2 # retorna el valor r1 = doble_print(5) # imprime 10, r1 = None r2 = doble_return(5) # r2 =  
10
```

Scope de variables: las variables definidas dentro de una función son locales — no existen fuera. Para modificar una variable global dentro de una función se usa la palabra clave `global`, aunque se recomienda evitarlo.

¿Para qué sirve el docstring (las comillas triples)? Documenta qué hace la función. Se accede con `help(funcion)`. Es una buena práctica profesional obligatoria en el curso.

Capítulo 5: Listas

Una lista es una colección ordenada y mutable de elementos. Los índices empiezan en 0. El índice -1 accede al último elemento.

```
numeros = [1, 2, 3, 4, 5] print(numeros[0]) # 1 – primer elemento print(numeros[-1]) # 5 –  
último elemento print(numeros[1:3]) # [2, 3] – slicing # Métodos principales  
numeros.append(6) # agrega al final numeros.insert(0, 0) # inserta en posición  
numeros.pop() # elimina y retorna último numeros.remove(3) # elimina el valor 3  
numeros.sort() # ordena en su lugar len(numeros) # longitud
```

IndexError: list index out of range — ocurre cuando se accede a un índice que no existe. Si lista tiene 5 elementos, los índices válidos son 0 a 4. lista[5] genera IndexError.

Mutabilidad: las listas son mutables. Si pasas una lista a una función, los cambios dentro de la función afectan la lista original. Esto es diferente a int y str que son inmutables.

```
Comprensión de listas – forma pythónica de crear listas: cuadrados = [x**2 for x in  
range(1, 6)] pares = [x for x in range(10) if x % 2 == 0]
```

Capítulo 6: Strings

Un string es una secuencia inmutable de caracteres. Se puede indexar y hacer slicing igual que las listas.

```
texto = 'Python' print(texto[0]) # P print(texto[-1]) # n print(texto[::-1]) # nohtyP –  
invertir print(len(texto)) # 6 # Métodos principales print(texto.upper()) # PYTHON  
print(texto.lower()) # python print(texto.replace('P','J')) # Jython print('hola  
mundo'.split()) # ['hola', 'mundo'] print(', '.join(['a','b','c'])) # a,b,c # f-strings  
(recomendado) nombre = 'Luis' print(f'Hola {nombre}')
```

¿Qué hace `[::-1]` en un string? Es slicing con paso -1 — recorre el string de atrás hacia adelante. Se usa para invertir strings, listas y tuplas.

Capítulo 7: Diccionarios

Un diccionario almacena pares clave:valor. Las claves deben ser únicas e inmutables. Acceso en $O(1)$.

```
estudiante = {'nombre': 'Luis', 'nota': 85} print(estudiante['nombre']) # Luis
print(estudiante.get('email', 'N/A')) # N/A (sin error) # Verificar si existe if 'nota' in
estudiante: print(estudiante['nota']) # Iterar for clave, valor in estudiante.items():
print(f'{clave}: {valor}')
```

KeyError: ocurre cuando se accede a una clave que no existe con []. Usar `get()` con valor por defecto para evitarlo:
`dict.get('clave', valor_default)`

Capítulo 8: Recursión

Una función recursiva se llama a sí misma. Necesita SIEMPRE un caso base que detenga la recursión y un caso recursivo que reduzca el problema.

```
def factorial(n): if n == 0 or n == 1: # caso base return 1 return n * factorial(n - 1) #  
caso recursivo def fibonacci(n): if n <= 1: # caso base return n return fibonacci(n-1) +  
fibonacci(n-2)
```

RecursionError: maximum recursion depth exceeded — ocurre cuando no hay caso base o nunca se alcanza. Python tiene un límite de ~1000 llamadas recursivas.

Traza de factorial(3): factorial(3) llama factorial(2), que llama factorial(1) que retorna 1 (caso base). Luego factorial(2) retorna $2 \cdot 1 = 2$, factorial(3) retorna $3 \cdot 2 = 6$.

¿Cuándo usar recursión vs iteración? Recursión es elegante para problemas que se definen recursivamente (árboles, factorial, fibonacci). Para procesar listas grandes, la iteración es más eficiente en Python.

Capítulo 9: Manejo de Excepciones

Las excepciones permiten manejar errores sin detener el programa abruptamente.

```
try: numero = int(input('Número: ')) resultado = 10 / numero except ValueError: print('No es un número válido') except ZeroDivisionError: print('No se puede dividir entre cero') except Exception as e: print(f'Error inesperado: {e}') finally: print('Siempre se ejecuta')
```

Excepciones más comunes en Python: ValueError (conversión inválida), TypeError (tipos incompatibles), IndexError (índice fuera de rango), KeyError (clave no existe en dict), NameError (variable no definida), AttributeError (método no existe en el tipo), ZeroDivisionError (división entre cero), RecursionError (stack overflow recursivo).

Capítulo 10: Errores Frecuentes por Semestre — UPTC

Basado en el registro de errores de los cursos de Programación I y II de los últimos tres años.

Semestre 1 — Programación I:

1. Confundir print con return (68% de estudiantes cometen este error en las primeras 3 semanas). 2. Usar = en lugar de == en condicionales. 3. Olvidar los dos puntos al final de if, for, def, while. 4. Indentación inconsistente — mezclar tabs y espacios. 5. IndexError al recorrer listas con índices incorrectos.

Semestre 2 — Programación II:

1. Recursión sin caso base claro. 2. Mutabilidad de listas — modificar dentro de funciones sin intención. 3. Scope de variables — usar variable global sin declararla con global. 4. Comprensión de referencias vs copias de objetos. 5. Manejo incorrecto de excepciones — capturar Exception genérico sin especificar.

Recomendaciones pedagógicas documentadas:

Para print vs return: pedir al estudiante que guarde el resultado en una variable y lo use en otra operación — el error de None queda evidente inmediatamente. Para IndexError: hacer que el estudiante imprima len(lista) antes de acceder al índice. Para recursión: dibujar el árbol de llamadas en papel antes de codificar.